

Stochastic Linear Bandits An Empirical Study

Students: Elliott Vigier, Loïc Cabannes,

Lecturer: Claire Vernade

1 Problem 1

1.1 Linear Epsilon Greedy[5]

The benchmark evaluates **Linear Epsilon Greedy** with $\epsilon = 0.0$, $\epsilon = 0.1$, $\epsilon = 0.5$, and $\epsilon = 0.8$, as well as a uniform random policy (**Unif**), across dimensions $d = 5$, $d = 10$, and $d = 30$. The results show that smaller ϵ values, like $\epsilon = 0.1$, strike a good balance between exploration and exploitation, while larger ϵ values, such as $\epsilon = 0.8$, lead to higher cumulative regret due to too much exploration.

As the dimensionality d increases, learning the hidden parameter θ becomes more challenging, which results in higher cumulative regret for all policies. At $d = 10$, the advantage of using $\epsilon = 0.1$ over $\epsilon = 0.5$ is clear, with $\epsilon = 0.1$ performing better. However, at $d = 30$, the high dimensionality makes it harder for all policies, and the differences between ϵ values become less noticeable, showing the limits of static exploration strategies.

In summary, although it struggles as the dimensionality increases **LinEpsilonGreedy** with a small ϵ works well in lower-dimensional settings and can therefore represent a good baseline. .

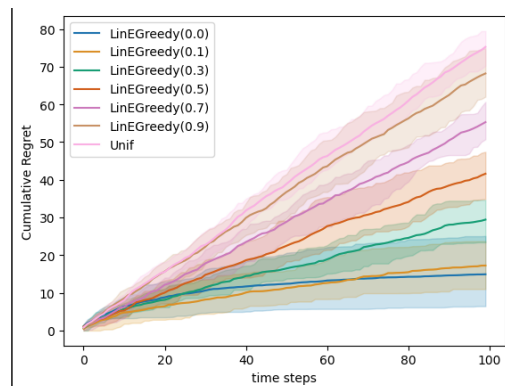


Figure 1: $d=3$

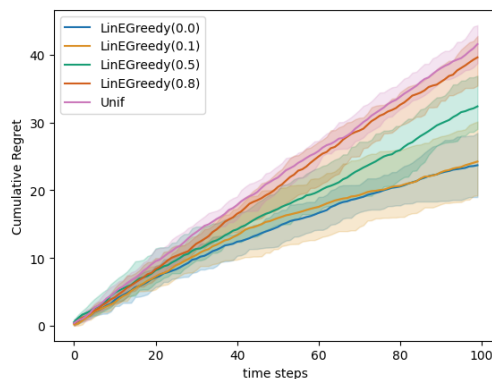


Figure 2: $d = 10$

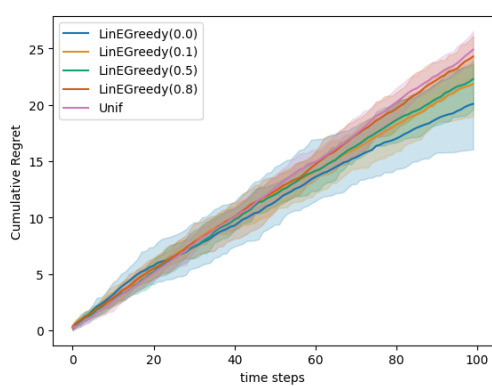


Figure 3: $d=30$

1.2 Matrix inversion step

Matrix inversion has a $O(d^3)$ complexity, where d is the dimension of the matrix being inverted (which corresponds to the dimensionality of the action space). This can become intractable, or at least computational demanding with large action spaces. Since we simply do a rank one update to the covariance matrix, we can use the Woodbury formula[3] (more specifically in our case of rank 1 updates, the Sherman-Morrison formula[4]) to efficiently update the inverse covariance matrix. The Sherman-Morrison Formula states that when doing an update $A_{t+1} = A_t + uv^T$, we can update its inverse in the following way:

$A_{t+1}^{-1} = \frac{A_t^{-1}uv^T A_t^{-1}}{1+v^T A_t^{-1}u}$ this update has a $O(d^2)$ complexity and can thus allow the algorithm to scale to much larger action spaces.

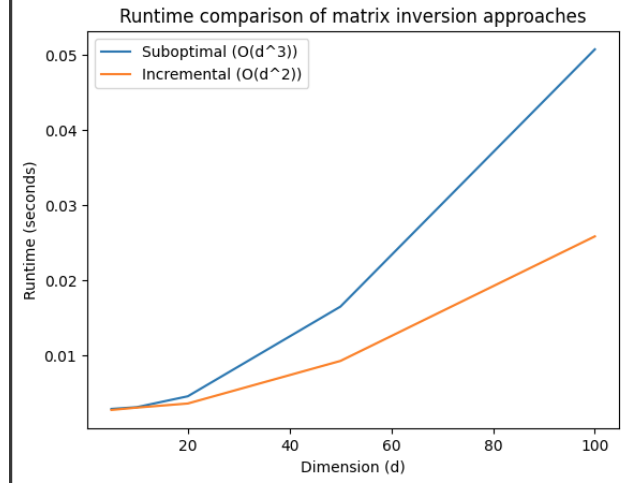


Figure 4: Runtime comparison of matrix inversion approaches for different values of d .

2 Problem 2: Linear UCB[2] and Linear Thomson Sampling[1] algorithms

2.1 Posterior at time t

Given a prior distribution $\pi_\theta(0) \sim \mathcal{N}(0, \lambda I)$, rewards $Y_t = A_t^T \theta + \epsilon$, with $\theta \sim \mathcal{N}(0, \lambda \mathcal{I})$
And, assuming that $\epsilon \sim \mathcal{N}(0, \sigma^2)$,

We get that the posterior at time t is also Gaussian:

$$\theta_t | A_1, \dots, A_t, Y_1, \dots, Y_t \sim \mathcal{N}(\mu_t, \Sigma_t)$$

$$\Sigma_t = \left(\frac{1}{\lambda} I + \frac{1}{\sigma^2} \sum_{1}^t A_t^T A_t \right)^{-1}$$

$$\mu_t = \Sigma_t \left(\frac{1}{\sigma^2} \sum_{1}^t A_t^T Y_t \right)$$

2.2 Comparison of algorithm performances

for clarity and computational reasons we restrict the algorithm we benchmark to linear epsilon greedy with $\epsilon=0$, as it was the best performing epsilon greedy policy we tested, linUCB, with σ set to 1 and $\delta \in \{0.1, 0.3, 0.5\}$, as well as Linear Thomson sampling with $\sigma = 1$ and $\lambda = 1$. To have higher confidence in the results, we greatly the number of roll-outs to 20 and set the time horizon to 100. Moreover, to test the robustness of the algorithms to environment, we measure performance on action space dimensionalities $d \in \{2, 5\}$ and fix the number of arms K to 5.

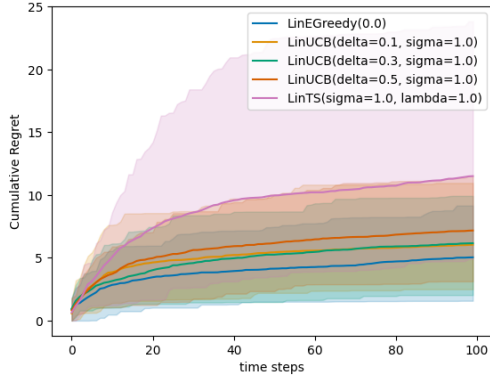


Figure 5: dimension=2

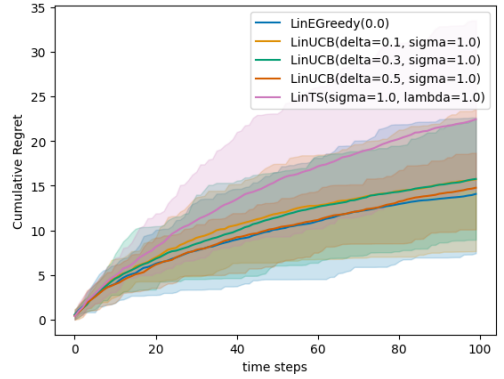


Figure 6: dimension = 5

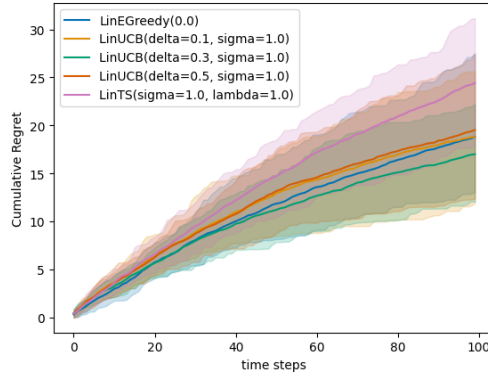


Figure 7: dimension=10

We can see that with a lower action space dimensionality, a greedy strategy outperforms other algorithms. However, as the number of dimensionality increases, linUCB matches, and then outperforms the greedy algorithm. Moreover, at least in these settings, although Linear Thomson Sampling also achieves logarithmic regret over time, it shows worse performs than any of the other algorithms.

From these results, we can conclude that in very low dimensionality settings a linear greedy strategy yields best performance, while in higher dimensionality settings the Linear UCB algorithm with the right hyper parameters outperforms other methods.

References

- [1] Shipra Agrawal and Navin Goyal. *Thompson Sampling for Contextual Bandits with Linear Payoffs*. 2014. arXiv: [1209.3352 \[cs.LG\]](https://arxiv.org/abs/1209.3352). URL: <https://arxiv.org/abs/1209.3352>.
- [2] Lihong Li et al. “A contextual-bandit approach to personalized news article recommendation”. In: *Proceedings of the 19th international conference on World wide web*. WWW ’10. ACM, Apr. 2010, pp. 661–670. DOI: [10.1145/1772690.1772758](https://doi.org/10.1145/1772690.1772758). URL: <http://dx.doi.org/10.1145/1772690.1772758>.
- [3] A Woodbury Max. “Inverting modified matrices”. In: *Memorandum Rept. 42, Statistical Research Group*. Princeton Univ., 1950, p. 4.
- [4] Jack Sherman and Winifred J. Morrison. “Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix”. In: *The Annals of Mathematical Statistics* 21.1 (1950), pp. 124–127. DOI: [10.1214/aoms/1177729893](https://doi.org/10.1214/aoms/1177729893). URL: <https://doi.org/10.1214/aoms/1177729893>.
- [5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.